

Script de démonstration — Réunion du mardi

selenium-bot : présentation + démo live (≈12-15 minutes avec la démo intégrale)

1. Introduction et contexte (≈45s)

À dire :

"Je vais vous présenter le bot que j'ai développé — **selenium-bot**. C'est un générateur de trafic web réaliste, construit avec Selenium et Java, dont le but est de simuler un utilisateur humain naviguant sur un site e-commerce.

L'idée derrière ce projet : tester si les outils classiques de détection réseau — comme Zeek et RITA — arrivent à distinguer ce trafic 'bot' d'un trafic humain normal. Si ces outils échouent, cela justifie la nécessité d'un Autoencoder — un modèle de machine learning — pour combler cette lacune. C'est la Phase 2 du projet."

Pas de commande ici — juste l'intro orale.

2. Architecture (≈1 min)

À dire :

"L'architecture repose sur 3 machines sur un réseau isolé :

- Un **Raspberry Pi** qui héberge un faux site e-commerce, `techmarket.lab` — un site statique de 14 pages, avec Nginx. C'est aussi sur ce Pi que tournent Zeek, pour capturer le trafic réseau, et RITA, pour l'analyse de type 'beacon'.
- Un **MacBook**, sur lequel tourne mon bot Selenium — c'est lui qui génère le trafic.
- Un **Dell**, qui sert de machine attaquante pour d'autres tests (Apache Bench, Slowloris) — pas utilisé dans cette démo.

Le bot ouvre un vrai navigateur Chrome headless et navigue sur le site comme le ferait un humain."

Commande à montrer (optionnel, si tu veux illustrer visuellement le site) :

```
open http://techmarket.lab/index.html
```

3. Le bot lui-même — comportement (≈1 min 15s)

À dire :

"Le bot simule 3 profils d'utilisateurs différents, avec des poids réalistes : 60% de navigation libre, 30% de recherche ciblée, 10% de remplissage de formulaire.

La navigation suit une chaîne de Markov — à chaque page, le bot choisit la suivante selon des probabilités définies. Et les temps d'attente entre chaque action ne sont pas aléatoires : ils sont calibrés statistiquement à partir du vrai jeu de données CICIDS2017, qui contient du trafic réseau humain réel. On utilise des distributions log-normales pour ça.

J'ai aussi un outil, `MixedTrafficRunner`, qui lance plusieurs bots en parallèle, avec des arrivées échelonnées dans le temps — pour simuler un site avec plusieurs visiteurs simultanés, comme un vrai site e-commerce."

Pas de commande ici — sauf si on veut montrer le code source rapidement :

```
open  
~/Desktop/selenium-bot/src/main/java/com/ids/bot/scenarios/BrowsingScenario.java
```

4. Démo live — Zeek (≈1 min)

À dire :

"Maintenant, montrons ce que Zeek voit réellement sur le réseau. Zeek capture chaque connexion et génère des logs structurés — ici, `conn.log`."

Commande à exécuter (SSH vers le Pi, afficher les dernières connexions) :

```
ssh raspberrypi@192.168.10.1 "sudo tail -15 /opt/zeek/spool/zeek/conn.log"
```

À dire pendant que ça s'affiche :

"Chaque ligne est une connexion TCP — IP source, IP destination, ports, durée, nombre de bytes échangés. C'est exactement le trafic que mon bot génère vers `techmarket.lab`, capturé de façon totalement transparente, sans aucune modification du bot."

Optionnel — si vous voulez aussi montrer le trafic HTTP (URL, méthode, status) :

```
ssh raspberrypi@192.168.10.1 "sudo tail -15 /opt/zeek/spool/zeek/http.log"
```

5. Démo live — RITA (≈1 min 30s)

À dire :

"Et maintenant, l'analyse RITA. RITA cherche des signatures de type 'beacon' — c'est-à-dire un trafic périodique, régulier, typique des malwares qui communiquent avec un serveur de commande et contrôle.

J'ai déjà importé une capture de mon trafic bot dans RITA. Regardons le score."

Commande à exécuter :

```
ssh -t raspberrypie@192.168.10.1 "cd /opt/rita && sudo docker compose run --rm rita view mixed_20260706_140847 --config /config.hjson"
```

(Le mot de passe SSH sera demandé — le taper à l'écran.)

À dire une fois le tableau affiché :

"Vous voyez ici le score de beacon pour la connexion vers `techmarket.lab` : **63.3%**, sévérité 'Low'. Ce qui est intéressant, c'est que ce score est quasiment identique à un test similaire fait la veille — donc le résultat est reproductible, ce n'est pas du bruit.

Mais surtout : ce score de 63.3% n'est ni clairement 'sain' ni clairement 'suspect'. Et si on compare avec un vrai humain naviguant lentement, testé plus tôt dans le projet, celui-ci obtient un score encore plus élevé — 89%. Donc RITA n'arrive pas à établir une frontière nette entre bot et humain sur ce type de trafic.

C'est exactement la preuve empirique qui justifie la Phase 2 : un outil classique comme RITA ne suffit pas ici, il faut un modèle apprenant — l'Autoencoder — pour aller plus loin."

6. Démo live intégrale — lancer le bot en direct (≈5-6 min)

C'est le moment fort : on ne montre plus une capture déjà faite, on lance le bot **en direct**, on observe Zeek pendant qu'il tourne, puis on regarde le score RITA à la fin. Prévoir **deux fenêtres de terminal côte à côte**.

À dire avant de lancer :

"Pour terminer, je vais lancer le bot en direct devant vous — pas une capture préparée à l'avance. Je vais utiliser `MixedTrafficRunner`, l'outil qui simule plusieurs visiteurs simultanés, pendant 5 minutes. Dans un second terminal, on va observer Zeek capturer le trafic en temps réel, connexion par connexion."

Terminal 1 — lancer le bot (Mac) :

```
cd ~/Desktop/selenium-bot  
mvn exec:java -Dexec.mainClass=com.ids.bot.MixedTrafficRunner -Dexec.args="5"
```

Terminal 2 — observer Zeek en direct (Pi), à lancer juste après :

```
ssh raspberrypie@192.168.10.1 "sudo tail -f /opt/zeek/spool/zeek/http.log"
```

À dire pendant que ça tourne (les deux terminaux affichent du texte en direct) :

"Vous voyez à gauche le bot qui lance des sessions — 'browsing', 'searching', 'form_filling' — de façon aléatoire mais pondérée. Et à droite, en temps réel, chaque requête HTTP que ce bot envoie apparaît dans les logs Zeek, avec l'URL exacte, la méthode, le user-agent. C'est la preuve que le trafic généré est un vrai trafic réseau, capturé de façon complètement passive."

Quand le bot affiche MIXED TRAFFIC SUMMARY (fin des 5 minutes) : arrêter Zeek avec `Ctrl+C` dans le Terminal 2, puis capturer le pcap et l'importer dans RITA pour avoir un score frais

:

```
# Récupérer le pcap généré (si capturé via capture_multiple_runs.sh en
parallèle)
# – sinon, utiliser le dataset déjà importé mixed_20260706_140847 comme montré
ci-dessous
ssh -t raspberrypie@192.168.10.1 "cd /opt/rita && sudo docker compose run --rm
rita view mixed_20260706_140847 --config /config.hjson"
```

Attention pratique : si tu veux un score RITA *vraiment* frais sur cette session de 5 minutes précise, il faut avoir lancé `tcpdump` sur le Pi en parallèle du bot (comme dans le protocole habituel), puis importer ce nouveau pcap dans RITA avant la réunion — ça prend quelques minutes de traitement. Si tu n'as pas le temps de préparer ça, réutilise simplement le dataset `mixed_20260706_140847` déjà importé (résultat quasi identique, déjà expliqué en section 5) et dis clairement que c'est une capture d'une session équivalente faite plus tôt, pas de cette session précise — reste honnête sur ce point devant Gérard.

7. Explication technique approfondie du bot (≈3-4 min)

Cette section sert à répondre en profondeur si on te demande "comment as-tu construit ça exactement" — ou à l'inclure proactivement si tu sens que l'audience est technique et intéressée par la méthodologie.

À dire :

"Je vais maintenant expliquer la logique derrière la construction du bot — d'où viennent les paramètres, pas juste ce qu'il fait.

Point de départ : les données. J'ai utilisé le jeu de données CICIDS2017, plus précisément le trafic BENIGN du lundi — environ 530 000 flux réseau, dont j'ai gardé environ 192 000 flux HTTP/HTTPS après nettoyage des valeurs infinies et aberrantes.

Étape 1 — Clustering. J'ai appliqué un KMeans avec $k=2$ sur ces flux, et le score de silhouette (0.556) confirme que 2 groupes est la structure naturelle des données — pas plus. Ces deux groupes correspondent à deux comportements distincts : un groupe avec des cycles actif/inactif longs et fréquents (91.5% du temps) — que j'ai appelé 'Browsing', navigation détendue ; et un groupe plus direct, avec seulement 16.4% de cycles actif/inactif — 'Searching', un utilisateur qui cherche quelque chose de précis.

Étape 2 — Modélisation statistique. Pour chaque groupe, j'ai analysé la distribution des temps d'attente. Ces temps suivent une distribution log-normale — typique en réseau, parce que la plupart des actions sont rapides mais il y a une longue traîne d'actions lentes. J'utilise la transformation $\log_{1p}$ pour gérer les valeurs proches de zéro correctement.

Mais il y a une subtilité : environ 67% des flux n'ont PAS de cycle actif/inactif du tout — ils passent directement à l'attente suivante. J'ai donc utilisé un modèle 'zero-inflated' : une porte de probabilité (Bernoulli) qui décide si on a un cycle actif/inactif, ou juste une attente simple.

Étape 3 — Le troisième profil, FormFilling. Ce profil n'est pas dérivé du clustering

— les données CICIDS ne contiennent pas d'utilisateurs remplissant des formulaires de façon distincte. Je l'ai construit en réutilisant les paramètres de 'Searching', mais avec une probabilité de cycle actif/inactif encore plus faible (5% au lieu de 16%), pour représenter un utilisateur très ciblé qui va droit au formulaire de contact.

Étape 4 — La navigation elle-même. Une fois les temps d'attente calibrés, il fallait aussi simuler QUELLE page le bot visite ensuite. Pour ça, j'utilise une chaîne de Markov — le prochain état ne dépend que de l'état actuel, avec des probabilités de transition. Il faut être honnête ici : ces probabilités de navigation ne viennent PAS de CICIDS — ce jeu de données ne contient pas d'information sur quelle URL a été visitée après quelle autre. Elles sont basées sur une hypothèse comportementale raisonnable, et je le documente explicitement dans le code.

Étape 5 — La calibration du débit d'arrivée. Pour simuler plusieurs visiteurs simultanés de façon réaliste, j'ai dû résoudre un problème de théorie des files d'attente. J'ai utilisé la loi de Little : $L = \lambda \times W$. Si le débit d'arrivée de nouveaux bots est trop rapide par rapport à la capacité de traitement, la file d'attente grandit sans limite. J'ai calibré l'intervalle d'arrivée pour obtenir un ratio $\rho \approx 0.68$ — stable, sans accumulation.

Donc en résumé : chaque paramètre de timing a une origine statistique traçable dans les vraies données CICIDS, sauf les probabilités de navigation entre pages, qui sont une hypothèse déclarée. Cette distinction entre 'dérivé des données' et 'hypothèse comportementale' est importante — je ne veux pas prétendre que tout est empiriquement fondé quand ce n'est pas le cas."

Pas de commande nécessaire ici — c'est une explication orale. Si on te demande une preuve visuelle, tu peux ouvrir le notebook d'analyse :

```
open ~/Desktop/IDS-Analysis/ANALYS-BENIGN/monday_analysis.ipynb
```

8. Conclusion / transition (≈15-20s)

À dire :

"Donc pour résumer : le bot est statistiquement validé, il produit un trafic dont la distribution correspond à du trafic humain réel, et les outils classiques comme RITA ne le distinguent pas de façon fiable. La prochaine étape est l'extraction de features depuis les logs Zeek, et l'entraînement de l'Autoencoder."

Récapitulatif des commandes (à avoir sous la main)

```
# 1. Voir le site (optionnel, visuel)
open http://techmarket.lab/index.html

# 2. Voir le code du bot (optionnel)
open
~/Desktop/selenium-bot/src/main/java/com/ids/bot/scenarios/BrowsingScenario.java

# 3. Zeek — connexions réseau brutes (aperçu statique, section 4)
ssh raspberrypi@192.168.10.1 "sudo tail -15 /opt/zeek/spool/zeek/conn.log"
```

```
# 4. Zeek – requêtes HTTP (optionnel, section 4)
ssh raspberrypi@192.168.10.1 "sudo tail -15 /opt/zeek/spool/zeek/http.log"

# 5. RITA – score de beacon (dataset déjà importé, section 5)
ssh -t raspberrypi@192.168.10.1 "cd /opt/rita && sudo docker compose run --rm
rita view mixed_20260706_140847 --config /config.hjson"

# 6. Démo live intégrale (section 6) – Terminal 1 : lancer le bot
cd ~/Desktop/selenium-bot && mvn exec:java
-Dexec.mainClass=com.ids.bot.MixedTrafficRunner -Dexec.args="5"

# 6. Démo live intégrale (section 6) – Terminal 2 : Zeek en direct
ssh raspberrypi@192.168.10.1 "sudo tail -f /opt/zeek/spool/zeek/http.log"

# 7. Notebook d'analyse CICIDS (optionnel, section 7)
open ~/Desktop/IDS-Analysis/ANALYS-BENIGN/monday_analysis.ipynb
```

Note pratique : teste toutes ces commandes une fois avant la réunion — en particulier vérifier que le Pi répond bien, que le dataset `mixed_20260706_140847` est toujours dans RITA (`rita list` pour vérifier), et que la démo live (section 6) fonctionne bien de bout en bout en conditions réelles (Chrome headless qui se lance correctement, SSH qui ne redemande pas le mot de passe au milieu).

Minutage total révisé : avec l'ajout des sections 6 et 7, la démo complète dure maintenant environ **12-15 minutes**, pas 5. Si le temps en réunion est vraiment limité à 5 minutes, privilégie les sections 1-3 et 5 (score RITA déjà prêt), et garde 6-7 en réserve si Gérard pose des questions ou demande à voir plus.